

Etape 1: Importation & chargement de dataset

```
In [457... import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, precision_score, recall_score
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
import matplotlib.pyplot as plt
import seaborn as sns
```

les Information sur cette dataset

```
In [458... df=pd.read_csv("Employeeetest.csv")
df.head()
```

```
Out[458...
```

	Education	JoiningYear	City	PaymentTier	Age	Gender	EverBenched	ExperienceInCurrentDomain	LeaveOrNot
0	Bachelors	2017	Bangalore	3	34	Male	No	0	0
1	Bachelors	2013	Pune	1	28	Female	No	3	1
2	Bachelors	2014	New Delhi	3	38	Female	No	2	0
3	Masters	2016	Bangalore	3	27	Male	No	5	1
4	Masters	2017	Pune	3	24	Male	Yes	2	1

```
In [459... df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4653 entries, 0 to 4652
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Education                             4653 non-null   object
1   JoiningYear                           4653 non-null   int64
2   City                                  4653 non-null   object
3   PaymentTier                           4653 non-null   int64
4   Age                                   4653 non-null   int64
5   Gender                                4653 non-null   object
6   EverBenched                           4653 non-null   object
7   ExperienceInCurrentDomain              4653 non-null   int64
8   LeaveOrNot                            4653 non-null   int64
dtypes: int64(5), object(4)
memory usage: 327.3+ KB
```

In [460...

df.describe(include='all')

Out[460...

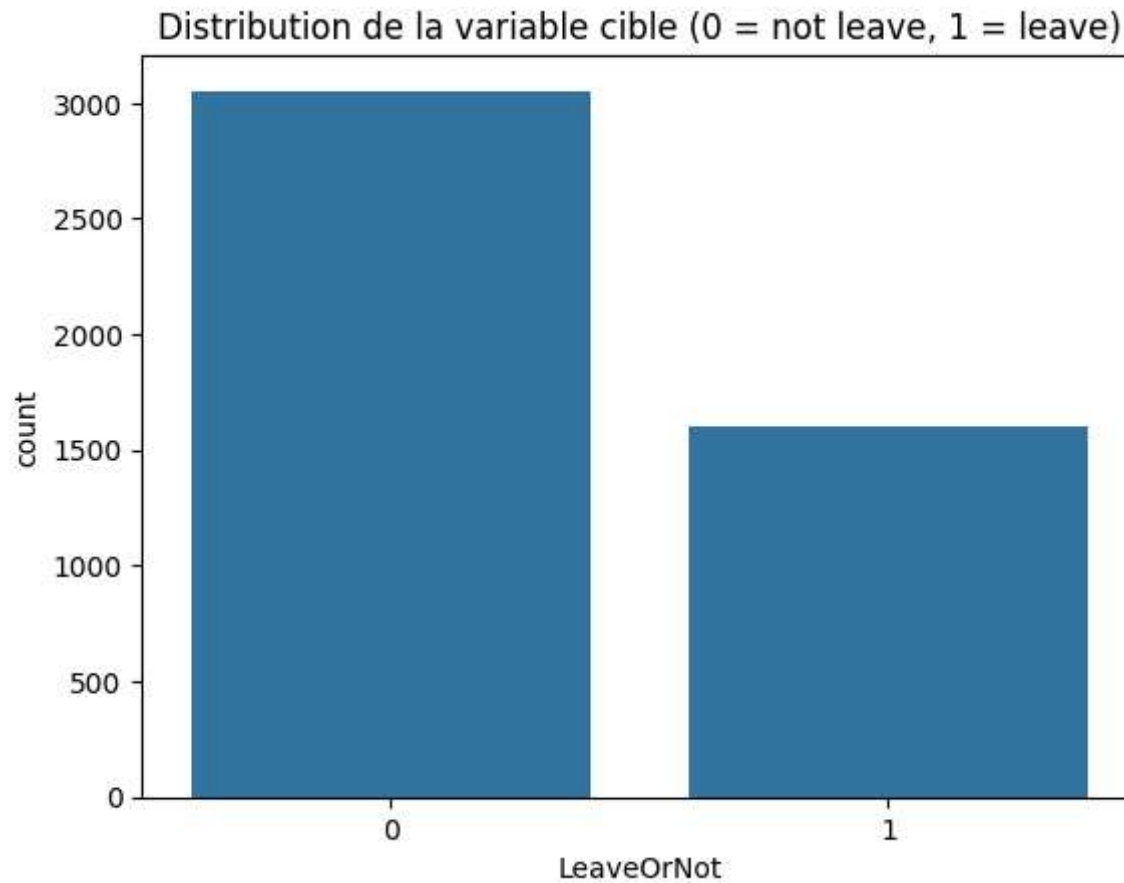
	Education	JoiningYear	City	PaymentTier	Age	Gender	EverBenched	ExperienceInCurrentDomain	LeaveOrNot
count	4653	4653.000000	4653	4653.000000	4653.000000	4653	4653	4653.000000	4653.000000
unique	3	NaN	3	NaN	NaN	2	2	NaN	NaN
top	Bachelors	NaN	Bangalore	NaN	NaN	Male	No	NaN	NaN
freq	3601	NaN	2228	NaN	NaN	2778	4175	NaN	NaN
mean	NaN	2015.062970	NaN	2.698259	29.393295	NaN	NaN	2.905652	0.343864
std	NaN	1.863377	NaN	0.561435	4.826087	NaN	NaN	1.558240	0.475047
min	NaN	2012.000000	NaN	1.000000	22.000000	NaN	NaN	0.000000	0.000000
25%	NaN	2013.000000	NaN	3.000000	26.000000	NaN	NaN	2.000000	0.000000
50%	NaN	2015.000000	NaN	3.000000	28.000000	NaN	NaN	3.000000	0.000000
75%	NaN	2017.000000	NaN	3.000000	32.000000	NaN	NaN	4.000000	1.000000
max	NaN	2018.000000	NaN	3.000000	41.000000	NaN	NaN	7.000000	1.000000

```
In [461... df.isnull().sum()
```

```
Out[461... Education          0
JoiningYear          0
City                0
PaymentTier          0
Age                 0
Gender              0
EverBenched         0
ExperienceInCurrentDomain  0
LeaveOrNot           0
dtype: int64
```

Distribution de la variable cible

```
In [462... sns.countplot(x='LeaveOrNot', data=df)
plt.title("Distribution de la variable cible (0 = not leave, 1 = leave)")
plt.show()
```



Etape 2:Transformation

In [463...

```
# colonne 'Education'
print(df['Education'].value_counts())
print("=====")
# colonne 'City'
print(df['City'].value_counts())
print("=====")
# colonne 'Gender'
print(df['Gender'].value_counts())
print("=====")
```

```
# colonne 'EverBenched'
print(df['EverBenched'].value_counts())
print("=====")
```

```
Education
Bachelors    3601
Masters       873
PHD           179
Name: count, dtype: int64
=====
City
Bangalore    2228
Pune         1268
New Delhi    1157
Name: count, dtype: int64
=====
Gender
Male         2778
Female       1875
Name: count, dtype: int64
=====
EverBenched
No           4175
Yes          478
Name: count, dtype: int64
=====
```

Faire LabelEncoder() et OneHotEncoder()

```
In [464... # Les colonne non numerique:
obj_col=['Education','Gender','EverBenched']
le=LabelEncoder()
for col in obj_col:
    df[col]=le.fit_transform(df[col])

# OneHotEncoder pour City
ohe = OneHotEncoder(sparse_output=False)
city_encoded = ohe.fit_transform(df[['City']])
city_cols = ohe.get_feature_names_out(['City'])
df_city_ohe = pd.DataFrame(city_encoded, columns=city_cols)
```

```
# Fusionner avec df
df = pd.concat([df.drop(columns=['City']), df_city_ohe], axis=1)
df.head()
```

Out[464...

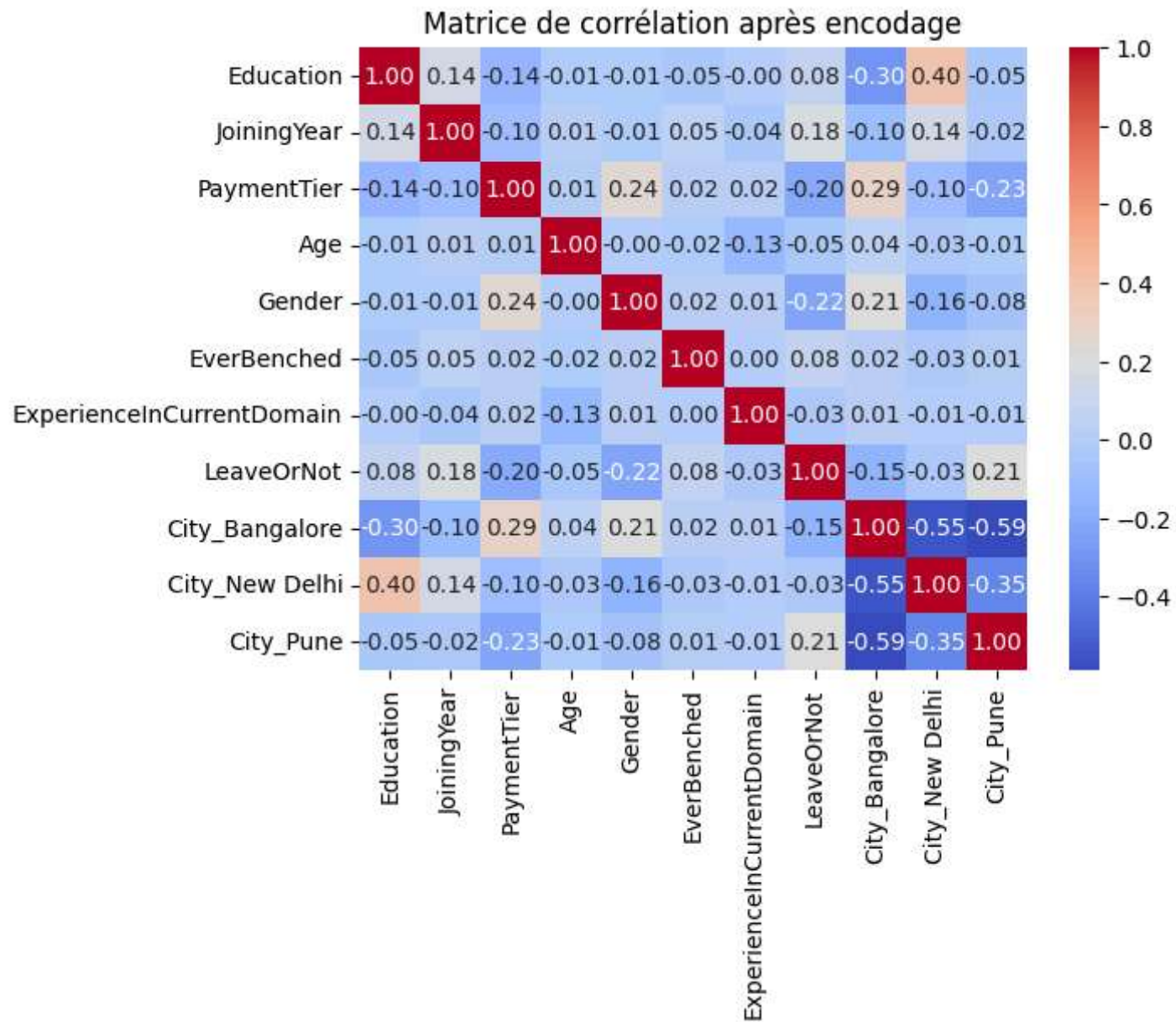
	Education	JoiningYear	PaymentTier	Age	Gender	EverBenched	ExperienceInCurrentDomain	LeaveOrNot	City_Bangalore	City_New Delhi
0	0	2017	3	34	1	0	0	0	1.0	0.0
1	0	2013	1	28	0	0	3	1	0.0	0.0
2	0	2014	3	38	0	0	2	0	0.0	1.0
3	1	2016	3	27	1	0	5	1	1.0	0.0
4	1	2017	3	24	1	1	2	1	0.0	0.0



Matrice de corrélation (avant encodage / après encodage)

In [465...

```
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Matrice de corrélation après encodage")
plt.show()
```



Etape 3: Split train/test

Separation des Donnes

```
In [466... # Separation de donnee:
X=df.drop(columns='LeaveOrNot')
X = np.hstack((np.ones((X.shape[0], 1)), X))
y=df['LeaveOrNot'].values.reshape(-1,1)
```

Standarisation de X

```
In [467... scaler=StandardScaler()
X=scaler.fit_transform(X)
```

Split les Donnees pour test et train

```
In [468... # Division:
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

Faire des fonction manuelle pour calculer cost

```
In [440... def sigmoid(z):
    return 1/(1+np.exp(-z))

def model_reg(X,theta):
    z=X.dot(theta)
    return sigmoid(z)

def fonction_objective(X,y,theta):
    m=len(y)
    y_pred=model_reg(X,theta)
    cost=-(1/m)*np.sum(y*np.log(y_pred)+(1-y)*np.log(1-y_pred))
    return cost

def gradient_opt(X,y,theta,learning_rate=0.01,iter=1000):
    m=len(y)
    cost_list=[]
```



```

for i in range(iter):
    y_pred=model_reg(X,theta)
    gradient=(1/m)*X.T.dot(y_pred-y)
    cost=fonction_objective(X,y,theta)
    theta-=learning_rate*gradient
    cost_list.append(cost)
    if (i%100==0):
        print(f"le cout apres iteratinos {i} est {cost}")
return theta,cost_list

# Entraînement manuel
#Train le medele:
theta_init=np.random.randn(X_train.shape[1],1)
theta_opt,cost_list=gradient_opt(X_train,y_train,theta_init,learning_rate=0.01,iter=10000)

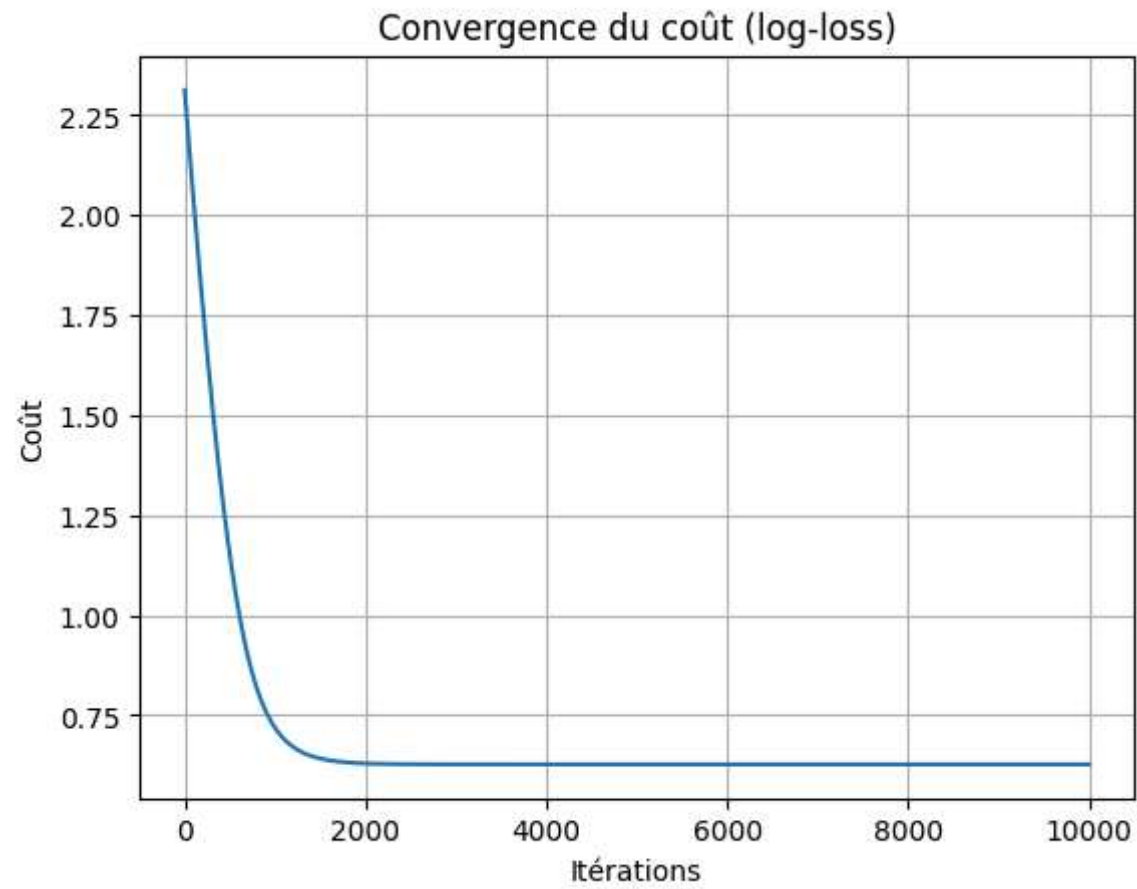
# Visualisation de la convergence
plt.plot(cost_list)
plt.title("Convergence du coût (log-loss)")
plt.xlabel("Itérations")
plt.ylabel("Coût")
plt.grid(True)
plt.show()

```

le cout apres iteratinos 0 est 2.3123612125944284
le cout apres iteratinos 100 est 2.0328573340053633
le cout apres iteratinos 200 est 1.7731793015884538
le cout apres iteratinos 300 est 1.536669434336797
le cout apres iteratinos 400 est 1.3276768471080256
le cout apres iteratinos 500 est 1.1505005420970769
le cout apres iteratinos 600 est 1.0074779280435413
le cout apres iteratinos 700 est 0.8975975568399458
le cout apres iteratinos 800 est 0.8166929392374337
le cout apres iteratinos 900 est 0.7589451113118738
le cout apres iteratinos 1000 est 0.7185092180329488
le cout apres iteratinos 1100 est 0.6904643292490117
le cout apres iteratinos 1200 est 0.6710755374006445
le cout apres iteratinos 1300 est 0.6576718568720628
le cout apres iteratinos 1400 est 0.6483980976238213
le cout apres iteratinos 1500 est 0.6419777725704324
le cout apres iteratinos 1600 est 0.6375320214081154
le cout apres iteratinos 1700 est 0.6344536565960192
le cout apres iteratinos 1800 est 0.6323220886080303
le cout apres iteratinos 1900 est 0.6308457295120643
le cout apres iteratinos 2000 est 0.6298225215639448
le cout apres iteratinos 2100 est 0.629112604719657
le cout apres iteratinos 2200 est 0.6286192897747455
le cout apres iteratinos 2300 est 0.6282757997039574
le cout apres iteratinos 2400 est 0.6280360437158827
le cout apres iteratinos 2500 est 0.6278682131851896
le cout apres iteratinos 2600 est 0.6277503479907218
le cout apres iteratinos 2700 est 0.6276672742070492
le cout apres iteratinos 2800 est 0.6276084929635168
le cout apres iteratinos 2900 est 0.6275667269964034
le cout apres iteratinos 3000 est 0.6275369208053635
le cout apres iteratinos 3100 est 0.627515553038384
le cout apres iteratinos 3200 est 0.6275001634931049
le cout apres iteratinos 3300 est 0.6274890275158397
le cout apres iteratinos 3400 est 0.6274809316017753
le cout apres iteratinos 3500 est 0.6274750184917843
le cout apres iteratinos 3600 est 0.6274706800253631
le cout apres iteratinos 3700 est 0.6274674828465969
le cout apres iteratinos 3800 est 0.6274651167454451
le cout apres iteratinos 3900 est 0.627463358624674
le cout apres iteratinos 4000 est 0.6274620472785543

le cout apres iteratinos 4100 est 0.627461065672589
le cout apres iteratinos 4200 est 0.6274603284431287
le cout apres iteratinos 4300 est 0.6274597730416421
le cout apres iteratinos 4400 est 0.6274593534330827
le cout apres iteratinos 4500 est 0.6274590355911347
le cout apres iteratinos 4600 est 0.6274587942628786
le cout apres iteratinos 4700 est 0.6274586106341571
le cout apres iteratinos 4800 est 0.6274584706369162
le cout apres iteratinos 4900 est 0.6274583637162353
le cout apres iteratinos 5000 est 0.6274582819280718
le cout apres iteratinos 5100 est 0.6274582192760575
le cout apres iteratinos 5200 est 0.6274581712219097
le cout apres iteratinos 5300 est 0.627458134322513
le cout apres iteratinos 5400 est 0.6274581059598473
le cout apres iteratinos 5500 est 0.627458084139268
le cout apres iteratinos 5600 est 0.6274580673383162
le cout apres iteratinos 5700 est 0.6274580543930425
le cout apres iteratinos 5800 est 0.627458044412278
le cout apres iteratinos 5900 est 0.6274580367128092
le cout apres iteratinos 6000 est 0.6274580307702344
le cout apres iteratinos 6100 est 0.6274580261816256
le cout apres iteratinos 6200 est 0.627458022637099
le cout apres iteratinos 6300 est 0.6274580198981311
le cout apres iteratinos 6400 est 0.6274580177809884
le cout apres iteratinos 6500 est 0.6274580161440495
le cout apres iteratinos 6600 est 0.6274580148780871
le cout apres iteratinos 6700 est 0.627458013898816
le cout apres iteratinos 6800 est 0.627458013141166
le cout apres iteratinos 6900 est 0.6274580125548817
le cout apres iteratinos 7000 est 0.6274580121011346
le cout apres iteratinos 7100 est 0.6274580117499158
le cout apres iteratinos 7200 est 0.6274580114780254
le cout apres iteratinos 7300 est 0.6274580112675235
le cout apres iteratinos 7400 est 0.6274580111045339
le cout apres iteratinos 7500 est 0.6274580109783224
le cout apres iteratinos 7600 est 0.6274580108805824
le cout apres iteratinos 7700 est 0.6274580108048863
le cout apres iteratinos 7800 est 0.6274580107462586
le cout apres iteratinos 7900 est 0.6274580107008485
le cout apres iteratinos 8000 est 0.6274580106656743
le cout apres iteratinos 8100 est 0.6274580106384274

le cout apres iteratinos 8200 est 0.6274580106173206
le cout apres iteratinos 8300 est 0.6274580106009698
le cout apres iteratinos 8400 est 0.6274580105883028
le cout apres iteratinos 8500 est 0.6274580105784892
le cout apres iteratinos 8600 est 0.6274580105708863
le cout apres iteratinos 8700 est 0.6274580105649958
le cout apres iteratinos 8800 est 0.627458010560432
le cout apres iteratinos 8900 est 0.6274580105568962
le cout apres iteratinos 9000 est 0.6274580105541565
le cout apres iteratinos 9100 est 0.6274580105520338
le cout apres iteratinos 9200 est 0.6274580105503891
le cout apres iteratinos 9300 est 0.6274580105491148
le cout apres iteratinos 9400 est 0.6274580105481273
le cout apres iteratinos 9500 est 0.6274580105473622
le cout apres iteratinos 9600 est 0.6274580105467693
le cout apres iteratinos 9700 est 0.62745801054631
le cout apres iteratinos 9800 est 0.6274580105459541
le cout apres iteratinos 9900 est 0.6274580105456783



```
In [452... y_pred=model_reg(X_test,theta_init)
y_pred=(y_pred>=0.5).astype(int)
performance=accuracy_score(y_test,y_pred)
print(f"accuracy_score:{performance*100:.4f}")
cr = classification_report(y_test, y_pred)
print(f"Classification Report :\n {cr}")
```

accuracy_score:66.0580

Classification Report :

	precision	recall	f1-score	support
0	0.79	0.65	0.72	610
1	0.51	0.67	0.58	321
accuracy			0.66	931
macro avg	0.65	0.66	0.65	931
weighted avg	0.69	0.66	0.67	931

```
In [472... model= LogisticRegression()
model.fit(X_train,y_train)
y_pred_sk = model.predict(X_test)
per = accuracy_score(y_test, y_pred_sk)
print(f"Accuracy score : {per *100:.4f}")
```

Accuracy score : 72.8249

c:\Users\pc\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\validation.py:1406: DataConversionWarning:
A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().
y = column_or_1d(y, warn=True)

```
In [474... import joblib
joblib.dump(model, 'regLog_model.pkl')
print("Modele sauvegarde sous le nom 'regLog_model.pkl')"
```

Modele sauvegarde sous le nom 'regLog_model.pkl'